SKYCOT PROTOCOL · ARCHITECTURE BRIEF

Multi-agent swarm coordination on a single git repo

A reusable protocol for coordinating a swarm of autonomous AI coding agents on one repository — using a shared ledger, no servers, and the rules of `git push`. Bootstrapped on the SKYCOT v1 build (now post-v1; ~800 PRs landed; the green capstone at PR #832).

Mark Hallam · Argo / SKYCOT · May 2026
Phase 2 revision — incorporates BUILD_PLAN_V11, the post-v1 strategic plan, and the tick-58 brief-authoring discipline lesson.

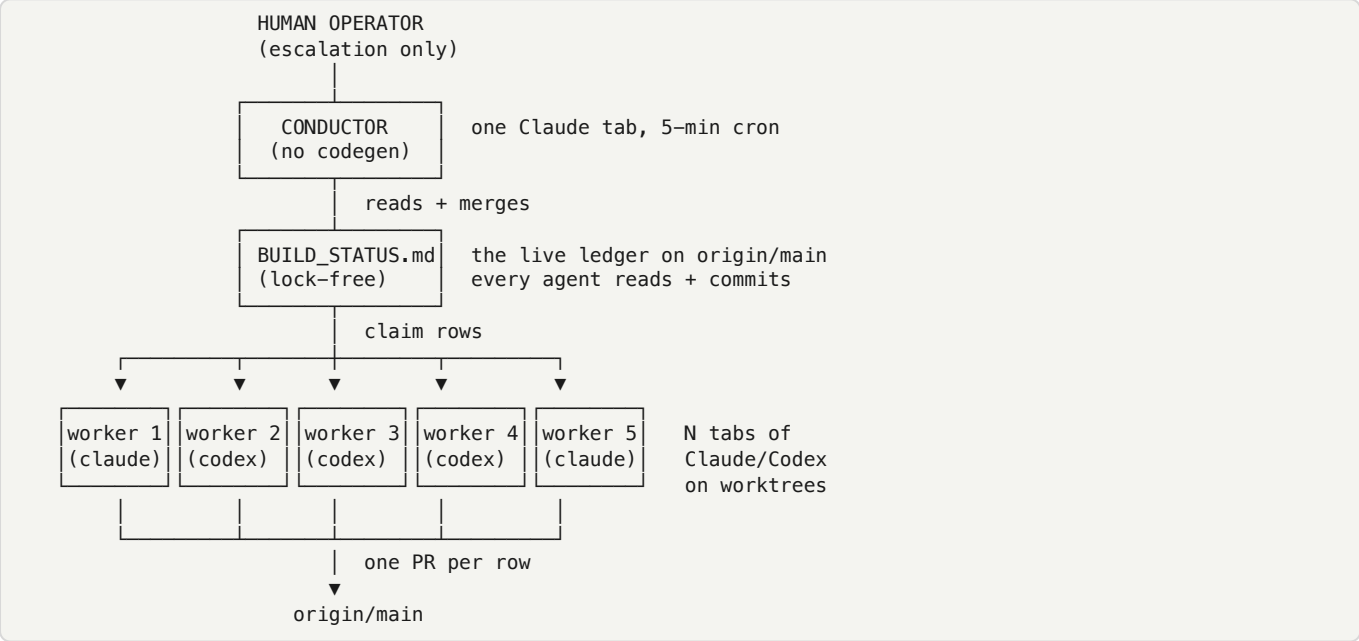
PROTOCOL — multi-agent swarm coordination

A reusable architecture for driving a multi-PR build with several AI coding agents working concurrently on the same git repo. Bootstrapped on the SKYCOT v1 build (22 brief slices, 21 merged PRs, ~60 minutes of wall time). This document is self-contained: someone reading it on a fresh project should be able to bootstrap with no prior context.

The companion artifacts:

- `PROTOCOL_LOOP_PROMPT.md` — the conductor-tick prompt to paste into `/loop 5m`.
- `PROTOCOL_BUILD_STATUS_TEMPLATE.md` — the seed ledger.
- `PROTOCOL_BRIEF_TEMPLATE.md` — the seed brief format.

Mental model



Three roles:

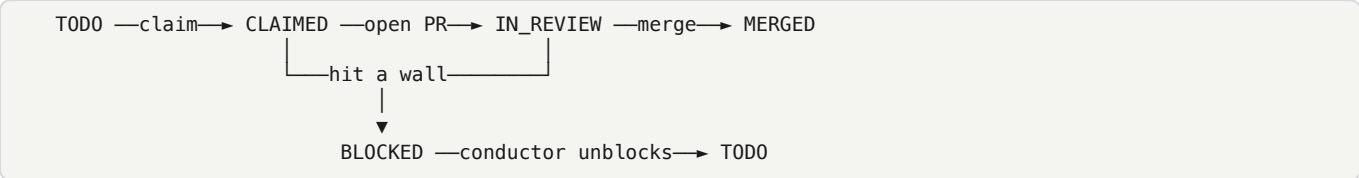
1. **Conductor** — ONE Claude tab. Driven by `/loop 5m <prompt>`. Never writes feature code. Pulls main, reviews open PRs, merges clean ones, updates the ledger, unblocks BLOCKED rows, supervises the swarm. The conductor is the only agent allowed to merge.
2. **Workers** — N tabs (mix Claude + Codex). Each tab claims one row from the ledger, implements it on a worktree, opens a PR, marks the row IN_REVIEW, then **stops**. Do not loop — the conductor does the merging and the human spawns a fresh worker tab when capacity opens up. (A self-pacing worker is fine too, but expect more conflict noise.)
3. **Human operator** — escalation target only. Reviews the rare row the conductor flags, resolves cross-stream architectural choices, kills runaway tabs.

Why two agent families? Claude and Codex make different mistakes; you get cheap diversity by running both. They share the same ledger but use distinct branch prefixes (`claude-*`, `codex-*`) so PR provenance is visible at a glance. Add a third family (Gemini, Cursor, etc.) later by reserving its own branch prefix.

The shared state is one file: `BUILD_STATUS.md`, in the repo, on `origin/main`. Every state transition is a one-line commit to that file. No database, no service — `git push` IS the lock. First push wins; losers pull and pick a different row.

The four state machines

1. Brief lifecycle



A row in BUILD_STATUS.md is in exactly one of these states:

- **TODO** — available to claim. Predecessors all MERGED.
- **CLAIMED** — a worker has the row. PR not yet open.
- **IN_REVIEW** — PR open. Conductor will review next tick.
- **MERGED** — landed on main. Successors may unblock.
- **BLOCKED** — worker hit a wall it cannot resolve unilaterally. Reason appended to the row. Conductor decides what to do next: expand the may-touch list, split the brief, or escalate to human.

The legal transitions are enforced by convention, not code. Workers pre-flight on `git pull --ff-only` before each transition; if push loses the race, pull and pick a different row.

2. Conductor tick (9 steps, 5-minute cron)

Every tick the conductor runs the same numbered loop. See `PROTOCOL_LOOP_PROMPT.md` for the exact prompt.

```
1. cd <repo> && git checkout main && git pull --ff-only
2. git log --oneline origin/main -8     (what landed since last tick)
3. gh pr list --state open               (what's in flight)
4. cat BUILD_STATUS.md | head -50        (queue state)
5. For each open PR from a worker branch you haven't reviewed:
   - gh pr diff <num> | head -300
   - Verify against the brief's acceptance checklist
   - Clean → squash-merge, delete branch, update ledger row
   - Issues → gh pr comment with the specific problem
6. Continue any conductor-owned in-progress work (rare)
7. For each BLOCKED row: investigate, expand may-touch, split, or
   escalate
8. Queue replenishment: count owner.TODO; if either ≤3 brainstorm
   3-5 new briefs and push as one commit per owner. See
   "Queue replenishment" below for details.
9. If nothing to do, report briefly + end the tick
```

Two rules that matter:

- **Never write feature code from the conductor tab.** It edits only BUILD_STATUS.md (status updates) and merges PRs. Mixing the conductor with a worker has bitten the swarm twice — see the webinbox anti-pattern below.
- **Always read from a fresh `git pull` of origin/main, not a stale local copy.** Workers are pushing to main constantly. The conductor tab's local working tree is allowed to be slightly stale; it just must `git pull --ff-only` at the top of every tick before reading or committing anything.

3. Worker tick

```
1. git checkout main && git pull --ff-only
2. cat BUILD_STATUS.md
3. Find the first row where:
     owner matches my agent family
     status == TODO
     all predecessors MERGED
4. Edit BUILD_STATUS.md → set that row's status to CLAIMED
5. git commit -m "status: claim <brief-id>" && git push origin main
   (if push fails: pull, pick a different row, retry)
6. Read streams/<brief-id>.md
7. git worktree add /tmp/<brief-id> origin/main
8. cd /tmp/<brief-id> && git checkout -b <owner>-<brief-id>
9. Implement the brief. Stay inside the may-touch list.
10. cargo build / clippy / test (or project equivalent). Tick the
    acceptance boxes. Verify file LOC budget.
11. git push -u origin <owner>-<brief-id>
12. gh pr create with the brief's PR title, body listing checked
    acceptance boxes
13. Edit BUILD_STATUS.md (in the main checkout, not the worktree)
    → status = IN_REVIEW, add PR number
14. git commit -m "status: <brief-id> in review (#NNN)" && git push
15. STOP. The conductor merges. A fresh tab starts the next brief.
```

The single most important worker rule: **one PR per tab, then stop**. Don't loop. Don't claim a second row from the same tab. The conductor has visibility on what's mergeable; the worker doesn't.

4. The unblock dance

When a worker hits a wall (clippy fails, the brief's may-touch list is too tight, or it discovers it needs to edit a workspace `Cargo.toml` that the brief forbade):

```
worker: edit BUILD_STATUS.md row to BLOCKED with reason
worker: git commit "status: <brief-id> blocked" && push
worker: pick a different brief, or stop
                  │
        next tick │
                  ▼
conductor: see BLOCKED row in step 7 of the tick
conductor: read the reason, decide:
            (a) expand brief's may-touch list (most common — Cargo.lock,
                workspace Cargo.toml deps, sibling crate one-line edit)
            (b) split brief into smaller sub-briefs (H-style)
            (c) flag for human (mission disagreement, contract change)
conductor: edit brief + BUILD_STATUS.md, commit
                  │
        next tick │
                  ▼
worker tab (fresh): see row back at TODO with expanded scope, claim
```

A typical block-unblock cycle is one or two ticks. The conductor's unblock commits during this run included:

- `status: unblock C-mcp-wire (Cargo.toml dep add now allowed)`
- `status: unblock C-mcp-wire (Cargo.lock now in may-touch list)`
- `status: unblock all 4 D-* briefs (lib.rs one-line module add allowed)`
- `status: D-push unblocked (use branch codex-D-push-v2)`
- `status: unblock C-gmail (Cargo.lock + defer keyring); apply same Cargo.lock allowance to all 5 C-* connector briefs`

These are the seams the brief author didn't anticipate. Expect one per stream.

Why it works

First-commit-wins on BUILD_STATUS.md is sufficient. Two workers that both pull main, both read TODO row X, both edit the row to CLAIMED, both `git push`: the second one's push fails fast-forward. That worker pulls, sees the row is now CLAIMED by the other, and picks a different row. No mutex, no service, no state machine beyond `git push`. We hit

this race ~5 times in 60 minutes of swarm. Cost: one extra pull per loser. Free correctness.

The file partition + per-brief may-touch list prevents most conflicts. Each brief lists "files you may touch" and "files you must NOT touch". Different streams get different files. The hot path of conflicts is a small set of shared seams (Cargo.toml / Cargo.lock / lib.rs / mod.rs / BUILD_STATUS.md), and those are either:

- explicitly opened in the brief (one-line module add, narrow dep add), or
- merged by the conductor manually when two PRs both touch them (3-way merge in a worktree, ~30 seconds).

The brief author over-restricts on purpose; the conductor unblocks when a worker bumps into something the brief forgot. Better than a silently-too-permissive brief that lets workers stomp each other.

Sub-slicing kept the integration capstone tractable. Stream H (auto-approval + orchestrator + skill library) was three coupled deliverables. Splitting into H-1 / H-2 / H-3 / H-4a / H-4b / H-4c let one worker land each independently — five PRs in series in one tab while the conductor merged each before the next started. The ledger row tracked the slice list: `H-1 ✓ #796 + H-2 ✓ #800 + H-3 ✓ #799 + H-4a ✓ #801 + H-4b ✓ #803`. A monolithic H brief would have stalled.

No central registry of agent identity. `git config user.email` and the `Co-Authored-By` trailer carry the provenance. Branch prefixes (`claude-*`, `codex-*`) carry the family. The owner column in BUILD_STATUS.md says who is supposed to pick the row. That's it.

Queue replenishment — keeping the swarm fed indefinitely

The original SKYCOT plan had a finite queue: 22 briefs across the 14-day v1. The conductor tick as originally written merged PRs and unblocked rows but never wrote new ones. Once the last brief was MERGED, every worker tab would idle and the swarm would die.

Queue replenishment fixes that. Step 8 of the conductor tick runs every cycle and asks one question per owner: is the TODO queue shallow? If yes, the conductor brainstorms a fresh batch of briefs from the project's mission and pushes them in one commit. The swarm self-extends.

The trigger fired naturally for the first time when `claude_todo` dropped to 0 — the conductor wrote 4 new claude briefs (claude-cu-skill-learn-wire, claude-tui-h-stack-panels, claude-e2e-integration-test, claude-genesis-llm-refinement) and pushed them as a single commit. The queue refilled in one tick.

The trigger

```
[every conductor tick]
        │
        ▼
count(owner.TODO)
        │
        ├── ≤ 3  → brainstorm + write 3–5 briefs → push one commit
        └── > 3  → skip
```

Concretely, at step 8 the conductor runs:

```
codex_todo  = grep -c "| codex |.*| TODO |"  <PLAN_DIR>/BUILD_STATUS.md
claude_todo = grep -c "| claude |.*| TODO |" <PLAN_DIR>/BUILD_STATUS.md
```

and fires the replenishment branch for each owner whose count is ≤3. Both queues above 3 → the step is a no-op.

Idea sources

When the trigger fires, the conductor brainstorms from four sources:

1. **`docs/skycot/MISSION.md`** (or whatever your project's mission doc is) — the long arc of the product. Pick a capability that is not yet in main.
2. **Recent merged-PR bodies' "deferred to follow-up" sections.** Workers routinely flag work they couldn't fit in their brief; the conductor mines those for ready-to-go briefs.

3. **The gap between the v1 demo and current main.** What does the demo claim that the code doesn't yet do? Each gap = one brief.
4. `git log --oneline origin/main -100` — recent commits often imply natural follow-ups (a new module that needs a test crate, a CLI that needs a `--json` flag, a metric that needs a reset).

Sizing — codex vs claude

The two agent families have different strengths. Briefs target the strength of their owner.

Owner	LOC budget	Scope	Acceptance flavor
codex	200–1200	Single-crate, narrow may-touch list	Mechanical, well-defined; mirror existing C/D/H briefs
claude	300–2000	May span crates; architectural decisions OK	Integration tests welcome; mirror existing claude-* briefs

Both follow `docs/build-plan/PROTOCOL_BRIEF_TEMPLATE.md`. Both go into `streams/<brief-id>.md` and get a row added to `BUILD_STATUS.md`.

The push convention

One commit per owner per replenishment tick. Subject:

```
briefs: replenish <owner> queue (<id1>, <id2>, <id3>, ...)
```

Example: `briefs: replenish claude queue (claude-cu-skill-learn-wire, claude-tui-h-stack-panels, claude-e2e-integration-test, claude-genesis-llm-refinement)`.

The commit edits `BUILD_STATUS.md` (adding new TODO rows) and adds the brief files in `streams/`. Workers see the new rows on their next pull.

Stop condition

Replenishment is unbounded by default. The conductor cancels the loop only when **every brief in BUILD_STATUS.md is MERGED AND the v1 demo gate is green** — for SKYCOT that means `claude-e2e-integration-test` passing on main and the demo script runnable end-to-end. This represents v1 customer-launch readiness; the conductor knows it's safe to stop because the product can ship.

If the human operator wants to stop earlier, ask the conductor in the chat — the loop responds to a kill instruction at the top of the next tick.

Anti-patterns we hit and how we recovered

These are real bugs from the SKYCOT run. Expect them on any swarm.

Two parallel agents claimed the same row

`status: claim C-mcp-wire` appears 5 times in the log (`a87db42`, `088436b`, `9f4c691`, `55a583c`, …). Each time, the second worker saw the row was now CLAIMED on its next pull, abandoned, and picked a different row. **Recovery: free, no human action needed.** First commit wins.

Two parallel agents implemented the same H sub-slice

Both #796 and a duplicate were opened against H-1. Conductor closed the duplicate on review (kept the more comprehensive PR). **Recovery: conductor's discretion in step 5 of the tick. 30 seconds.** Lesson: when conductor approves the claim of a multi- slice brief, it should immediately update the ledger row to list the sub-slices it expects, so workers don't double-claim within the brief.

`Cargo.toml` `workspace.members` conflicts every time a new crate landed

Every Stream D channel + Stream C connector added a new crate. Each PR edited workspace `members = [...]`. PRs merged in arbitrary order; second-to-merge always conflicted on that line. **Recovery: conductor opens a worktree on origin/main, runs `git rebase origin/main` on the conflicting branch, hand-resolves the workspace list, force-with-leases the branch, and re-merges.** ~30 seconds per conflict. Five conflicts total over the run.

Fix for next time: lock workspace.members in a brief, OR have the conductor pre-add empty crate dirs at the start of the run so worker PRs only add files to existing crates.

A conductor-tick commit accidentally swept up a worker's WIP from the working tree

The webinbox bug. Worker had unstaged webinbox files in the conductor's working tree. The conductor ran `git commit -am "status: …"` with `-a` (which stages all tracked-file modifications). The status-update commit shipped 7 untracked-file additions. The worker came back, saw its files already on main, and the open PR became redundant. **Recovery: the row was marked MERGED with a note explaining what happened; the open PR was closed as redundant (content was identical to main).**

Fix: **always run the conductor tab from a separate worktree on origin/main, never the main checkout.** Or use `git add <file>` + `git commit -m` (not `-am`) for status updates. The `PROTOCOL_LOOP_PROMPT.md` explicitly stages only BUILD_STATUS.md.

Branch contention — codex-D-push was stale

A codex worker had opened PR #785 on branch `codex-D-push`. The branch went stale (rebased badly, conflicts piled up). Conductor's unblock: allow the worker to use `codex-D-push-v2` and close #785. **Recovery: one-line ledger update naming the new branch. Worker re-implemented from fresh main.**

Lesson: when a PR has been open >2 hours with no progress, the conductor should ping it; if no response in another tick, close the PR and let a fresh worker re-claim.

Brief shape didn't match reality (executor field schema)

Stream A locked the executor-field schema in the manifest. Stream C-mcp-wire later needed a related but different schema for `[mcp_servers]`. The original brief said "do NOT modify any existing declaration" in argo-vertical. C-mcp-wire blocked twice asking for permission to add a new section. **Recovery: conductor expanded the brief in two iterations** (`status: unblock C-mcp-wire (allow narrow argo-vertical [mcp_servers] schema add)`).

Lesson: contract briefs (Stream A here) define the trait/schema shape; downstream briefs that touch that crate get a "narrow add" clause for new sections. Bake into the template.

The roles, in detail

Conductor

- ONE Claude tab. Driven by `/loop 5m <PROTOCOL_LOOP_PROMPT.md>`.
- Allowed to: pull main, read BUILD_STATUS.md, read PR diffs, merge clean PRs, comment on PRs with issues, edit BUILD_STATUS.md (status transitions only), edit briefs in `streams/` (to expand may-touch lists when unblocking), close stale PRs, **write new briefs when a TODO queue runs shallow** (queue replenishment — see section above).
- NOT allowed to: write feature code, modify other files, modify workspace deps, push to anything other than main.
- Tick budget: ~3 minutes per tick. The 5-minute interval gives 2 minutes of slack; longer ticks usually mean the conductor got drawn into hand-fixing a merge conflict, which is fine occasionally but a smell if it becomes routine.
- Stop condition: every brief MERGED **and the v1 demo gate is green** (e.g. the integration-test brief passes), or human intervention. Queue replenishment keeps the loop alive indefinitely otherwise.

Worker (Claude or Codex)

- One tab per brief. Stops after one PR.

- Allowed to: pull main, claim a TODO row, read its brief, implement on a worktree branch, push the branch, open a PR, mark the row IN_REVIEW. Mark BLOCKED with reason if stuck.
- NOT allowed to: merge anything, modify another brief's files, modify the conductor's BUILD_STATUS.md transitions out of order, delete or force-push another agent's branch.
- Stop after the IN_REVIEW commit. Do not loop.

Human operator

- Spawns worker tabs. Cancels runaway tabs. Reviews any row the conductor escalates.
- Resolves architectural decisions the conductor isn't authorized to make (e.g. mission-level pivots, security trade-offs, third-party API choices not pinned in the brief).
- Watches the ledger periodically. If the conductor is stuck in a loop (same BLOCKED row two ticks running), intervenes.

The artifacts

Three files in `docs/build-plan/` (or wherever your repo's plan lives). All committed to the repo, all read/written by every agent.

`BUILD_STATUS.md` — the ledger

Single file, one table, one row per brief. Plus three appendix sections: dependencies graph, blocked rows, done rows. Every state transition is a commit to this file. See `PROTOCOL_BUILD_STATUS_TEMPLATE.md` for the exact shape.

Commit-message convention is mandatory:

- `status: claim <brief-id>`
- `status: <brief-id> in review (#<PR-number>)`
- `status: <brief-id> merged (#<PR-number>)`
- `status: <brief-id> blocked`
- `status: unblock <brief-id> (<reason>)`

This makes `git log --oneline origin/main` a readable swarm timeline, which is what the conductor reads at step 2 of every tick.

Brief files in `streams/<brief-id>.md`

One brief per row in the ledger. The shape is fixed. See `PROTOCOL_BRIEF_TEMPLATE.md`. Required sections:

- Owner / Depends on
- Scope (1–2 paragraphs)
- Files you may touch
- Files you must NOT touch
- Acceptance criteria (checkboxes)
- Branch + PR title convention
- Notes for the implementer

The "may touch" list is the brief's most important section. Tight = fewer conflicts but more BLOCKED unblock cycles. Loose = more conflicts but smoother flow. Start tight; the conductor expands as unblocks happen.

`CONTRIBUTING_AGENT.md` — worker rules

The worker-side protocol. Each worker reads this once before starting. Covers: how to claim, the gates before opening a PR (build / lint / test / LOC budget), branch + PR conventions, what to do when stuck, the rules-that-override-everything list (no force-push to main, no editing other agents' branches, etc.).

The exact text used on SKYCOT lives at `docs/build-plan/CONTRIBUTING_AGENT.md` and is short enough (~140 lines) that workers can paste it into context every session.

The git workflow

Worktree-per-PR

Every worker creates its own worktree before touching code:

```
git worktree add /tmp/<brief-id> origin/main
cd /tmp/<brief-id>
git checkout -b <owner>-<brief-id>
# ... implement ...
```

Why: the conductor tab and other workers may all be checked out on the same repo's `main`. Separate worktrees mean separate working trees. No `git stash` dance, no accidental staging across briefs.

The conductor uses its own worktree too (`/tmp/conductor` or just its tab's main checkout, kept clean — never editing files there).

Direct-push to main for ledger updates only

Status updates are one-line edits to BUILD_STATUS.md. They do not go through PR review:

```
git checkout main
git pull --ff-only
# edit BUILD_STATUS.md
git add docs/build-plan/BUILD_STATUS.md
git commit -m "status: claim X"
git push origin main
```

Code changes always go through a PR. Only ledger transitions direct-push.

Squash-merge for code PRs

`gh pr merge <num> --squash --delete-branch`. The squashed commit title becomes the merge-commit subject; preserve the brief id in parens (e.g. `executor: trait + manifest enum (A-executor-trait)`). This makes `git log --oneline` itself a readable feature timeline.

Force-with-lease for rebase pushes

When the conductor hand-resolves a workspace.members conflict on a worker branch:

```
cd /tmp/<brief-id>
git fetch origin
git rebase origin/main
# resolve conflict in Cargo.toml
git rebase --continue
git push --force-with-lease origin <branch>
```

`--force-with-lease`, never plain `--force`. Protects against clobbering a worker that may have pushed something the conductor hadn't fetched yet.

Per-file 500-LOC budget

Hard constraint enforced by every worker's acceptance checklist and verified by the conductor on PR review. Encourages workers to split implementations across multiple sibling files (`worker_actions_executor.rs` + `worker_actions_executor_tests.rs`

- `worker_actions_dispatch.rs`, etc) rather than monolithing. Compounding effect: smaller diffs → faster review → less merge- conflict surface area.

Bootstrapping on a new project

1. Create `docs/build-plan/` (or your project's equivalent).

2. Copy `PROTOCOL.md`, `PROTOCOL_LOOP_PROMPT.md`, `PROTOCOL_BUILD_STATUS_TEMPLATE.md`, `PROTOCOL_BRIEF_TEMPLATE.md` into it.
3. Rename the templates: `PROTOCOL_BUILD_STATUS_TEMPLATE.md → BUILD_STATUS.md`, etc, and fill in your project's brief rows.
4. Write one brief per intended PR slice in `streams/<brief-id>.md`. Tighten may-touch lists; the conductor will expand later.
5. Write a tight `CONTRIBUTING_AGENT.md` adapting the worker rules to your toolchain (replace `cargo build/clippy/test` with your build commands, keep the LOC budget if you want it).
6. Open one Claude tab. Paste `PROTOCOL_LOOP_PROMPT.md` content into a `/loop 5m` command, substituting your repo path. Hit enter. The conductor is now running.
7. Open N worker tabs (mix of Claude and Codex). In each, paste: "Read `docs/build-plan/CONTRIBUTING_AGENT.md` and pick the first TODO row matching your agent family." Hit enter.
8. Watch the ledger. Spawn fresh worker tabs as ones finish.

That's it. The swarm is live.

Phase 2 — beyond the v1 demo gate

The protocol described above bootstrapped on a finite plan: 22 seed briefs → v1 demo gate. As of tick 57, that gate is **GREEN** (9/9 e2e steps passing; ~800+ PRs landed since the seed run; the green capstone landed at PR #832, `claude-e2e-test-step-completion`). The swarm did not stop. This section captures the shape of the protocol on the other side of the v1 gate — when the loop is no longer racing toward a fixed finish line, but is the standing substrate for the long-arc product mission.

BUILD_PLAN_V11 — the strategic plan beyond v1

At tick 57 the conductor authored `BUILD_PLAN_V11.md` alongside `BUILD_STATUS.md`. Where BUILD_STATUS.md is the live ledger (rows + state transitions, edited every tick), BUILD_PLAN_V11.md is the **strategic horizon** — the multi-month phasing of work between "v1 demo green" and the full mission claims.

It does three things:

1. **Recapitulates the mission claims** as load-bearing bullets (real teammates · platform-built workers · shared skills library · self-learning skills · persistent memory · multi-channel approvals).
2. **Names the 7 gaps** between v1's demo-real implementation and a production-real platform (reliability · capability breadth · operator visibility · multi-worker orchestration · self-improvement · org awareness · demo verticals).
3. **Phases the work** into seven labelled stages:

```
v1.1  Production hardening   (reliability the operator can trust)
v1.2  Capability breadth     ("any PC/Mac task")
v1.3  Multi-worker DAGs      (teams of workers)
v1.4  Operator UX            (non-terminal operators)
v2.0  Self-improvement       (workers actually get better)
v2.1  Governance             (enterprise + regulated)
v2.2  Demo verticals         (prove >1 use case)
```

The Step-8 source list, post-v1

Queue replenishment (Step 8 of the conductor tick) was originally sourced from `MISSION.md`, recent PR-body deferrals, the v1-demo gap list, and `git log -100`. After tick 57, it adds a fifth source:

5. **`BUILD_PLAN_V11.md` phased plan** — when either TODO queue dips ≤3, the conductor reads the phase that matches "what's active right now" (initially v1.1; later v1.2+v1.3 in parallel) and turns one or two un-shipped bullets into briefs.

The trigger and stop-condition are unchanged. The replenishment loop is the same one. Only the idea sources are richer.

Sequencing rules between phases

The phases are **not** strictly serial. The plan-V11 sequencing rules:

- **v1.1 first.** Reliability is the precondition for everything else. Without operator trust, capability and breadth do not matter.
- **v1.2 and v1.3 run in parallel.** More executor kinds + more connectors is additive and orthogonal to multi-worker orchestration.
- **v1.4 is a long tail.** Operator UX work is large and ongoing; start with the high-impact piece (web dashboard) early, let the rest land incrementally.
- **v2.0 and v2.1 wait on v1.1 stable.** Self-improvement and governance assume the production-reliability foundation is solid.
- **v2.2 (more demo verticals) is the customer signal.** Each new vertical is one more data point for the platform claim "any role."

What does NOT change post-v1

- **The stop condition.** Still "every brief MERGED **and** the e2e gate is green." The gate stays the production-readiness oracle. New briefs land → new e2e steps land alongside them → the gate keeps the bar.
- **The conductor tick shape.** Same 9 steps, same 5-minute cadence. Step 8 just has a richer list to draw from.
- **The four state machines.** Brief lifecycle, conductor tick, worker tick, unblock dance — all unchanged.
- **The roles.** Conductor still doesn't write feature code. Workers still stop after one PR. Human still escalation-only.
- **The artifacts.** BUILD_STATUS.md still the ledger. Briefs still in `streams/`. CONTRIBUTING_AGENT.md still the worker-side rules.

The protocol survives the v1-to-v2 transition unchanged in shape; only the queue-source list grows.

Phase 2 in flight (snapshot, tick 58)

12 mission-gap briefs queued in `streams/` straight off BUILD_PLAN_V11 (6 codex + 6 claude). Spread across:

- **v1.1 reliability** — circuit-breaker, multi-step-approval, anomaly-detector
- **v1.2 capability** — stripe / jira / asana connectors, skill-marketplace-search, filesystem-executor
- **v1.3 multi-worker** — workflow-dag, org-awareness
- **v2.0 self-improve** — skill-versioning-ab-test
- **v2.2 demo verticals** — multi-vertical bundle

Standard replenishment cadence resumes once these clear. The swarm is now self-sustaining indefinitely against the strategic plan.

Sizing and pacing

A rough rule of thumb from the SKYCOT run:

- **Brief size**: 1 brief = ~1 PR = ~200–500 LOC = ~30–60 minutes of one worker tab.
- **Conductor cadence**: 5-minute tick fires ~12 times/hour. ~50% of ticks are no-ops ("nothing to merge, nothing to unblock"). ~30% merge one PR. ~15% merge multiple PRs or unblock a row. ~5% require human escalation.
- **Worker concurrency**: 4–6 active workers is a sweet spot for one human operator. More than 8 and the conductor's PR review queue starts to back up; the merge becomes the bottleneck.
- **Throughput on a clean run**: 1 PR every 3 minutes of wall time is achievable when the dependency graph is fanned-out (e.g. Stream C connectors fanning out off C-mcp-wire). Sustained throughput is closer to 1 PR every 5–8 minutes once you average in serial dependencies.

If the swarm stalls (no merges in two consecutive ticks, no open PRs, multiple BLOCKED rows): the conductor or human should cut a new tighter brief or split a fat one. The swarm runs on a steady diet of small briefs; starve it and it dies.

Appendix — Brief authoring discipline

A lesson from tick 58: **always ground the may-touch list in the real codebase before publishing a brief.**

Tick 57's first batch of 6 codex briefs ALL came back BLOCKED because the file paths were imagined rather than verified. The brief said `argo-runtime/`, but the crate is `argo-identity/`. The brief said `argo-mcp/src/connectors/`, but each connector is a separate `argo-mcp-connector-<name>` crate. The brief said `argo-skill-store`, but the crate is just `argo-skill`. Six workers wasted six claim cycles unblocking the same kind of mistake. The briefs were re-amended at tick 58 with corrected paths.

The fix is one shell command before publishing:

```
find crates -name '*.rs' | grep <thing>     # or
fd <thing> crates/                          # or
rg -l <symbol>
```

If you can't name the file with a `find`, you can't name it in the may-touch list. Workers will hit it and BLOCK. Better to spend 30 seconds verifying than to ship 6 dead briefs.

PROTOCOL_LOOP_PROMPT — the conductor-tick prompt

Paste the prompt below into `/loop 5m <prompt>` on the project repo. This drives the conductor tab. See `PROTOCOL.md` for the full architecture and `CONTRIBUTING_AGENT.md` for the worker-side rules.

How to run:

1. Substitute the four placeholders in angle brackets:
   - `<REPO_PATH>` — absolute path to the repo (e.g. `/Users/me/code/myproject`).
   - `<PLAN_DIR>` — relative path of the plan directory inside the repo (e.g. `docs/build-plan`).
   - `<WORKER_BRANCH_PREFIXES>` — comma-list of branch prefixes the swarm uses (e.g. `claude-,codex-`).
   - `<PROJECT_NICKNAME>` — what to call the swarm in the report (e.g. `SKYCOT 14-day v1`).
2. Open ONE Claude tab on `<REPO_PATH>`. This tab will be the conductor for the duration of the run.
3. Run `/loop 5m <paste the prompt below>`.
4. Cancel the loop when every brief in BUILD_STATUS.md is MERGED AND the v1 demo gate is green, or by Ctrl-C / human intervention.

The lines flagged `[PROJECT-SPECIFIC]` are the ones you'll change per project. Every other line is reusable verbatim.

Note on step 8 (queue replenishment): the prompt differentiates codex vs claude brief sizing. Codex briefs are smaller (200–1200 LOC), single-crate, mechanical, narrow may-touch. Claude briefs are larger (300–2000 LOC), can span crates, can make architectural decisions, can include integration tests. When the conductor brainstorms new briefs at step 8, it picks the sizing flavor based on which queue is short.

Post-v1 idea sources for step 8: once the v1 demo gate is green, queue replenishment also draws from `BUILD_PLAN_V11.md` (the phased plan from v1.1 production hardening through v2.2 demo verticals). The conductor reads the phase that matches "what's active right now" and turns un-shipped bullets into briefs. See `PROTOCOL.md` "Phase 2 — beyond the v1 demo gate" for the full source list.

```
5m Conductor tick for the <PROJECT_NICKNAME> swarm. Run this exact sequence:

1. cd <REPO_PATH> && git checkout main && git pull --ff-only origin main 2>&1 | tail -3

2. git log --oneline origin/main -8  (see what landed since last tick)

3. gh pr list --state open --json number,title,headRefName,author,createdAt 2>&1  (see open PRs)

4. cat <PLAN_DIR>/BUILD_STATUS.md | head -60  (see queue state)

5. For any NEW open PR from a <WORKER_BRANCH_PREFIXES> branch I haven't reviewed yet:
   - gh pr diff <num> --color never | head -300 (skim the diff)
   - Verify against the brief's acceptance checklist (file boundaries, LOC, clippy/test passing as claimed)
   - If clean: gh pr merge <num> --squash --delete-branch, then update BUILD_STATUS.md row to MERGED with PR
num, commit "status: <brief-id> merged (#<num>)", push to main
   - If issues: gh pr comment <num> --body "<specific issue>", note in BUILD_STATUS.md
   - Track which PRs I've already reviewed across ticks (use git log to see my own merge commits)

6. Continue any in-progress claude-owned Stream work I'm building. Pick the first claude-owned TODO row whose
predecessors are all MERGED.

7. If a brief has been BLOCKED, investigate the reason and decide: expand its may-touch list (small fix in
BUILD_STATUS.md + brief), or split into sub-briefs, or escalate to the human operator.

8. Queue replenishment:
   a. codex_todo = grep -c "| codex |.*| TODO |" <PLAN_DIR>/BUILD_STATUS.md
      claude_todo = grep -c "| claude |.*| TODO |" <PLAN_DIR>/BUILD_STATUS.md
   b. If codex_todo <= 3: brainstorm 3-5 NEW codex-sized briefs. Idea sources, in priority
      order: MISSION.md gaps, BUILD_PLAN_V11.md phased plan (the v1.x / v2.x stage that's
      currently active), recent PR-body "deferred to follow-up" sections, v1 demo gaps, and
      git log -100. Each brief follows PROTOCOL_BRIEF_TEMPLATE.md. Sizing: 200-1200 LOC,
      narrow may-touch, single-crate focus.
   c. If claude_todo <= 3: same for claude. Sizing: 300-2000 LOC, multi-crate OK,
      architectural decisions OK, integration tests welcome.
   d. **Brief authoring discipline (tick-58 lesson)**: before publishing a brief, ground
      the may-touch list in the real codebase. Run `find crates -name '*.rs' | grep <thing>`
      or `rg -l <symbol>` to verify every file path exists. Imagined paths cause BLOCKED
      claims — 30 seconds of verification beats six dead briefs.
   e. Push ONE commit per replenishment: "briefs: replenish <owner> queue (<id1>, <id2>, ...)".
   f. Skip if both above 3.

9. If everything is in flight and there's nothing to do, report back briefly with
   "swarm healthy, X PRs in review, Y briefs claimed, Z merged, codex_todo=A, claude_todo=B"
   and end the tick.

Do not stop the loop. Cancel only when the user asks to stop OR when every brief in BUILD_STATUS.md is MERGED
AND the v1 demo gate is green.
```

Notes

- The 5-minute interval is a default. For larger swarms (8+ workers) drop to 3 minutes. For sleepier projects (1–2 workers, hand-paced) raise to 10 minutes or run unscheduled.
- `head -8` on the git log and `head -300` on the diff are tuned for Claude's context budget per tick. Bump if you want deeper review.
- The conductor reports "nothing to do" cleanly — that's the expected state on most ticks. Don't fight it.
- If the conductor tab itself crashes / loses state, restart the loop in a fresh tab. The ledger on origin/main IS the state; the conductor's local memory is disposable.
- Step 8 (queue replenishment) is what keeps the swarm fed indefinitely. Without it, the loop runs out of work as soon as the seed briefs all merge. The trigger is simple: count TODO rows per owner, fire if either is ≤3, write 3–5 new briefs in one commit. See `PROTOCOL.md` "Queue replenishment" for the idea sources, sizing rules, and stop condition.
- Hard rules every tick (carry over from earlier protocol revisions): always run from `<REPO_PATH>` on the main branch after a fresh `git pull --ff-only`; never `git commit -am` (always stage explicit files); never force-push to main; never modify a worker's branch except via squash-merge or close.

PROTOCOL_BUILD_STATUS_TEMPLATE — seed BUILD_STATUS.md

Copy this file to `BUILD_STATUS.md` (in the same directory) on a new project, fill in the queue rows for your briefs, commit it to `main`. Every agent reads + writes this file from then on.

See `PROTOCOL.md` for the architecture, `PROTOCOL_LOOP_PROMPT.md` for the conductor's tick, and `PROTOCOL_BRIEF_TEMPLATE.md` for the brief shape.

```
# Build status — live ledger

**Updated by every agent on every state transition.** Always read
from `origin/main`; never trust a stale local copy.

Status values: `TODO` (claim me) · `CLAIMED` (someone is working) ·
`IN_REVIEW` (PR open) · `MERGED` (on main) · `BLOCKED` (needs
unblock).

When you claim or update a row, commit `BUILD_STATUS.md` with
message `status: <action> <brief-id>` (e.g. `status: claim
<brief-id>`, `status: <brief-id> in review (#<PR>)`,
`status: <brief-id> merged (#<PR>)`, `status: <brief-id> blocked`).

Stage **only** `BUILD_STATUS.md` (and, when unblocking, the brief
file) — never `git commit -am`. The conductor enforces this; a
sweep of the working tree has bitten the swarm before.

---

## Active queue

| Brief | Owner | Status | PR | Last update |
|---|---|---|---|---|
| <brief-id-1> | claude | TODO | — | (not yet started) |
| <brief-id-2> | codex | TODO | — | (waits on <brief-id-1>) |
| <brief-id-3> | codex | TODO | — | (waits on <brief-id-1>) |

Owner values: `claude`, `codex`, or your additional agent family
prefix. Conductor uses no owner column — it merges everyone's PRs.

---

## Picking your next brief

Workers pick the first row where:
- `owner == <my agent family>`
- `status == TODO`
- All "depends on" predecessors have `status == MERGED`

If multiple agents try to claim the same row, the one whose commit
lands first on main wins. The losers will see the row is already
CLAIMED on their next pull and pick a different one.

If no row matches, stop the tab. The conductor or human will spawn
fresh work.

---

## Known dependencies

Encode the dependency graph as ASCII art so workers can verify
predecessors at a glance:
```



(no dependencies) └──

```
The "depends on" column above is the source of truth; this graph
is illustrative.

---

## Blocked rows

(none currently)

When marking BLOCKED, append a short reason on the same row:
`status=BLOCKED reason="cargo clippy fails on field X; need
Stream Z's trait shape first"`.

Conductor's job at the next tick: read the reason, decide
(a) expand may-touch list, (b) split into sub-briefs, or
(c) escalate to human.

---

## Done rows

(none currently)

When the queue is fully MERGED, append a `Plan complete` note here
with the date and a link to the deliverable (demo recording,
release tag, postmortem doc).
```

Commit-message convention (mandatory)

Every status transition commits `BUILD_STATUS.md` with one of these exact verbs in the subject line:

- `status: claim <brief-id>` — TODO → CLAIMED
- `status: <brief-id> in review (#<PR>)` — CLAIMED → IN_REVIEW
- `status: <brief-id> merged (#<PR>)` — IN_REVIEW → MERGED (conductor)
- `status: <brief-id> blocked` — any → BLOCKED
- `status: unblock <brief-id> (<reason>)` — BLOCKED → TODO (conductor)
- `status: <brief-id> TODO (<reason>)` — re-opening, e.g. after PR closed

This makes `git log --oneline origin/main` itself a swarm timeline. The conductor reads this log at step 2 of every tick and uses it to figure out what landed since last tick.

Optional: counts at the top

For long-running plans, prepend a one-line summary above the queue table that the conductor updates periodically:

`**Progress:** 21 / 22 briefs MERGED · 1 CLAIMED (H-4c)`

This is purely informational. The queue table is the source of truth.

What NOT to do

- Don't put PR review notes in this file. They go in `gh pr comment` against the PR.
- Don't put architectural debate here. Use the brief's notes section, or escalate to human in the Blocked rows section.
- Don't track sub-task completion at the row level for fat briefs. Either the brief is small enough to be one row, or split it into separate rows (each its own brief file).

- Don't delete rows. Once a row exists, it ends in MERGED or as a documented closure note. The ledger is part of the audit trail.

PROTOCOL_BRIEF_TEMPLATE — seed brief

One brief per row in the ledger. One brief = one PR. Workers read the brief once and follow it exactly. The conductor reads the brief when reviewing the worker's PR (acceptance checklist) and when deciding how to unblock a BLOCKED row (may-touch expansion).

Save each brief as `streams/<brief-id>.md`. The `<brief-id>` should match the ledger row exactly (e.g. `A-executor-trait`, `C-gmail`, `H-4b`).

The template

Copy this verbatim, fill in the angle-bracket sections, delete this preamble:

```
# Stream <X> — <one-line summary>

**Owner**: <claude | codex | other agent family>.
**Depends on**: <brief-id-N merged | none>.
<optional: critical-path / capstone / preflight tag>

## Scope

<1–2 paragraphs describing what this PR delivers and why. State the
contract this PR is locking, if any (e.g. "this is a CONTRACT PR;
downstream briefs implement against the trait this PR defines —
get the shape right"). State explicitly what is NOT in scope but
might appear to be.>

## Files you may touch

- `<crate/path/to/file.rs>` — <what changes>
- `<crate/path/to/file_tests.rs>` — <what changes>
- `<NEW: crate/path/to/new_file.rs>` — (NEW)
- `<Cargo.toml>` — narrowly scoped: <add ONE dependency line for X.
  No other edits.>
- `<crate/lib.rs>` — narrowly scoped: <add ONE module line. No
  other edits.>

## Files you must NOT touch

- <crate or directory you must stay out of, with a one-line reason
  per entry — usually "owned by Stream Y" or "lock from brief Z">
- Anything not on the may-touch list above

If your task genuinely requires a file not listed, **stop and mark
this brief BLOCKED in BUILD_STATUS.md** with a one-line reason. Do
not silently expand scope.

## Acceptance criteria

- [ ] <checkbox describing one verifiable outcome — type, function,
      test, or behavioral assertion. Each box must be independently
      verifiable from a fresh checkout.>
- [ ] <next box>
- [ ] <gate: project's build command passes>   e.g. `cargo build -p <crate>`
- [ ] <gate: lint command passes>               e.g. `cargo clippy -- -D warnings`
- [ ] <gate: tests pass>                        e.g. `cargo test -p <crate>`
- [ ] Every modified file under <LOC budget>    e.g. < 500 LOC
- [ ] BUILD_STATUS.md updated with PR number

## Branch + PR

- Branch: `<owner>-<brief-id>` — e.g. `claude-A-executor-trait`
- PR title: `<area>: <one-line summary> (<brief-id>)`

## Notes for the implementer

- <implementation hints, gotchas, library choices, naming
  conventions specific to this brief>
- <any locked decisions: "use library X, not Y", "match the
  existing pattern in file Z", "do not introduce a new abstraction
  here">
- <any expected edge cases the worker should test>
```

Sizing rules

- **One brief = one reviewable PR.** If you can't review the PR in 10 minutes, the brief was too big. Split.
- **Target 200–500 LOC per brief.** Briefs above 500 LOC tend to hit conflicts mid-implementation. Briefs below 100 LOC are fine but indicate the dependency graph could be coarser.
- **Acceptance criteria: 5–12 boxes.** Fewer than 5 means the brief is under-specified; more than 12 means split.
- **The may-touch list should fit on one screen.** If it doesn't, the brief is touching too much. Split.

If a brief turns out fat in flight, the worker marks BLOCKED with reason `too large, request split`. The conductor then sub-slices the brief into N sub-briefs, each its own row in the ledger (e.g. `H-1`, `H-2`, `H-3`, `H-4a`, `H-4b`, `H-4c`).

Locking the contract

When a brief defines a shared abstraction (a trait, a schema, an event type) that downstream briefs depend on, the brief should say so explicitly:

> This is a CONTRACT PR. Other streams will implement against the trait + tests this PR defines. Get the shape right.

Downstream briefs that need to extend the contract (add a new section to a schema, add a new enum variant) get a "narrowly scoped" allowance in their may-touch list:

> `<file in contract crate>` — narrowly scoped: add a NEW `<NewSection>` (struct + re-export) for the `[new_section]` table. Do NOT modify any existing declaration in those files.

This pattern keeps the contract crate stable while allowing forward-compatible additions.

What a good brief looks like (anonymised example)

A small example brief, sized like Stream C's connector briefs from the SKYCOT run. Project name replaced with `<vertical>`, `<connector>` as substitutions.

```
# Stream C — <connector> MCP connector

**Owner**: codex. **Depends on**: C-mcp-wire merged.

## Scope

Implement an MCP server (stdio JSON-RPC) that exposes <connector>
operations as tools the agent can call mid-turn. The connector lives
in its own crate `<vertical>-<connector>`, registers with the agent
runner via the existing `[mcp_servers]` manifest section, and uses
the connection pool wired in C-mcp-wire.

This brief follows the shape locked by C-gmail (the first connector
to land). Reuse the OAuth refresh + token cache pattern from
`<vertical>-gmail/src/oauth.rs`; do not invent a new one.

## Files you may touch

- `crates/<vertical>-<connector>/**` (NEW crate)
- `Cargo.toml` — narrowly scoped: add ONE workspace member entry
  `crates/<vertical>-<connector>`. No other workspace edits.
- `Cargo.lock` (auto-updated by cargo when adding the crate)
- `manifests/connectors/<connector>.toml` (config: api token,
  base url, default scopes)

## Files you must NOT touch

- `crates/<vertical>-mcp/**` (the runtime — locked from C-mcp-wire)
- Any other connector crate
- `crates/<vertical>-vertical/**` (manifest schema — locked from A)
- The `Executor` trait

## Acceptance criteria

- [ ] Crate `<vertical>-<connector>` builds standalone
- [ ] Exposes 5 tools: `search`, `read`, `create`, `update`,
      `<one connector-specific tool>`
- [ ] OAuth refresh path: token cache at
      `<cwd>/.argo/connectors/<connector>/`; refresh token rotation
      on 401
- [ ] Integration test gated on `<CONNECTOR>_TEST_TOKEN` env var
- [ ] Manifest registration in `manifests/connectors/<connector>.toml`
- [ ] `cargo clippy --workspace --all-targets -- -D warnings` clean
- [ ] `cargo test -p <vertical>-<connector>` passes
- [ ] Every file < 500 LOC

## Branch + PR

- Branch: `codex-C-<connector>`
- PR title: `mcp(connector): <connector> — search / read / create / update / <X> (C-<connector>)`

## Notes for the implementer

- The OAuth pattern from C-gmail is the template. Match it exactly,
  including the env-var names and cache directory layout.
- Don't introduce a new HTTP client — reuse `reqwest` with the
  workspace-pinned version.
- If the <connector> SDK has poor docs, prefer raw HTTP calls.
  Don't pull in a new SDK crate without flagging in BUILD_STATUS.
- The agent dispatch path (C-mcp-wire) auto-discovers tools via
  `list_tools()`; just implement the JSON-RPC handler. No
  registration boilerplate beyond the manifest entry.
```

This is a real-shaped brief: ~50 lines, ~9 acceptance boxes, tight may-touch list, explicit reuse of an upstream pattern, an explicit non-goal ("don't introduce a new HTTP client"). A worker can land this in one PR in one tab.
